

Compact Flash Card Reader

2024/08/15

Karunarathna K.W.T.D.

Muhammed M.I.

Contents

1	CF Card Specification.....	5
1.1	Block Diagram	5
1.2	CompactFlash Storage Card Physical Specification	5
1.3	Pin Configuration.....	7
1.4	True IDE Mode I/O Function	8
1.5	True IDE Mode Addressing.....	8
1.6	Register Configuration.....	10
2	CF Card Reader Board	12
2.1	Block Diagram	12
2.2	Schematic Diagram	13
2.3	PCB Design	14
2.4	Printed Circuit Board.....	15
3	FAT 32 File Structure	16
3.1	Master Boot Record	22
3.2	Volume ID of the first partition.	25
3.3	Necessary Calculations.....	28
3.4	How to locate a file in the Root Directory.....	30
3.5	How to locate a Sub-Directory (folder) in the Root Directory.....	33
3.6	How to locate a file in the Sub-Directory in the Root Directory	36
4	Observation.....	38
5	References.....	39

List of Figures

Figure 1: CompactFlash Storage Card Block Diagram	5
Figure 2: CompactFlash Storage Card Dimensions	6
Figure 3: 50-Pin Connector	6
Figure 4: True IDE Mode I/O Function	8
Figure 5: True IDE Mode I/O Decoding	9
Figure 6: Data Register Access	10
Figure 7: Drive/Head Register	10
Figure 8: Status & Alternate Status Register	10
Figure 9: Read Sector(s)	11
Figure 10: Block Diagram of CF Card Reader.....	12
Figure 11: Schematic Diagram of CF Card Reader Board	13
Figure 12: PCB Design of CF Card Reader Board	14
Figure 13: CF Card Reader Board with CF Card	15
Figure 14: FAT 32 File System Structure	16
Figure 15: Master Boot Record	16
Figure 16: Partition Entry	17
Figure 17: FAT32 Volume ID, critical fields	17
Figure 18: Critical Fields of Volume ID and values	17
Figure 19: Important Variables for Reading Fat-32	18
Figure 20: FAT32 Filesystem Overall Layout	18
Figure 21: Formula to turn cluster number into the LBA address	19
Figure 22: 32 Byte Directory Structure, Short Filename Format	19
Figure 23: File Allocation Table and Following Cluster Chains	21
Figure 24: MBR Structure.....	22
Figure 25: Partition Entry Structure	22
Figure 26: Master Boot Record Layout.....	23
Figure 27: Partition_1 Entry	24
Figure 28: Boot Sector Structure.....	25
Figure 29: Volume ID (Boot Sector).....	26
Figure 30: Fat 32 Volume ID, Critical Fields.....	27
Figure 31: Sector Locations	29
Figure 32: Directory Entry of the File ABC	30
Figure 33: 32-byte Directory Layout	30
Figure 34: Location of Cluster 8	31
Figure 35: First Sector of the File ABC.....	32
Figure 36: Files and Folders in CF Card.....	33
Figure 37: Directory Entry of the Folder "LE Robotics" in the Root Directory	34
Figure 38: Layout of the Directory Entry of the Folder "LE Robotics"	34
Figure 39: Sub Directory of the Folder LE Robotics.....	35
Figure 40: Sub Directory Entry of the File "About LE Robotics"	36
Figure 41: Layout of the Directory Entry of the File "About LE Robotics"	36
Figure 42: First Sector of the File "About LE Robotics"	37
Figure 43: Observed Output from Oscilloscope.....	38

List of Tables

Table 1: CompactFlash Storage Card Physical Specifications	5
Table 2: Pin Assignments and Pin Type of Cf card In True IDE Mode	7
Table 3: Field, Offset and Size of the 32-byte Directory	20
Table 4: Attribute Byte Bit Functions in Directory Records	20
Table 5: Master Boot Record Data	23
Table 6: Partition Entry_1 Data	24
Table 7: Fat 32 Volume ID and Critical Fields Data	27
Table 8: Data of the Directory Entry of the File ABC	30
Table 9: Data of the Directory Entry of the Folder "LE Robotics"	34
Table 10: Data of the Directory Entry of the Folder "LE Robotics"	36

1 CF Card Specification

1.1 Block Diagram

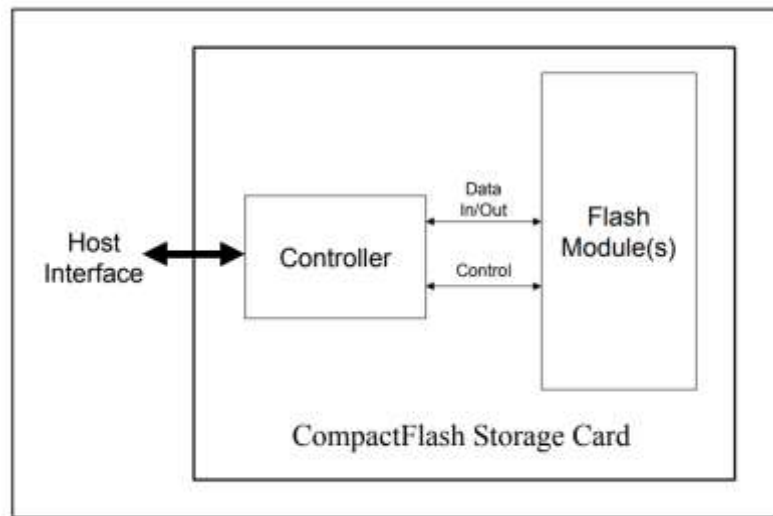


Figure 1: CompactFlash Storage Card Block Diagram

1.2 CompactFlash Storage Card Physical Specification

Table 1: CompactFlash Storage Card Physical Specifications

Length	36.4 ± 0.15 mm (1.433 ± .006 in.)
Width	42.80 ± 0.10 mm (1.685 ± .004 in.)
Thickness (Including Label Area)	3.3 mm ± 0.10 mm (.130 ± .004 in.)

1.3 Pin Configuration

Table 2: Pin Assignments and Pin Type of Cf card In True IDE Mode

Pin Number(s)	Signal Name	Pin Type
18, 19, 20	A2 - A0	Input
31, 30, 29, 28, 27, 49, 48, 47, 6, 5, 4, 3, 2, 23, 22, 21	D15 – D00	Input / Output
46	-PDIAG	Input / Output
45	-DASP	Input / Output
26, 25	-CD1, -CD2	Output
7, 32	-CS0, -CS1	Input
39	-CSEL	Input
1, 50	GND	-
34	-IORD	Input
35	-IOWR	Input
9	-ATA SEL	Input
37	INTRQ	Output
44	Reserved (-REG)	Input
41	-RESET	Input
13, 38	VCC	-

1.4 True IDE Mode I/O Function

The CompactFlash Storage Card can be configured in a True IDE Mode of operation. The CompactFlash Storage Card is configured in this mode only when the -OE input signal is grounded by the host during the power off to power on cycle. Optionally, CompactFlash Storage Cards may support the following optional detection methods:

1. The card is permitted to monitor the -OE (-ATA SEL) signal at any time(s) and switch to PCMCIA mode upon detecting a high level on the pin.
2. The card is permitted to re-arbitrate the interface mode determination following a transition of the (-) RESET pin.
3. The card is permitted to monitor the -OE (-ATA SEL) signal at any time(s) and switch to True IDE mode upon detection of a continuous low level on a pin for an extended period of time.

Host implementers should not rely on any of these optional detection methods in their designs. In the True IDE Mode, the PCMCIA protocol and configuration are disabled and only I/O operations to the Task File and Data Register are allowed. In this mode, no Memory or Attribute Registers are accessible to the host. CompactFlash Storage Cards permit 8-bit data accesses if the user issues a Set Feature Command to put the CompactFlash Storage Card in 8-bit Mode.

Note: Removing and reinserting the CompactFlash Storage Card while the host computer's power is on will reconfigure the CompactFlash Storage Card to PC Card ATA mode from the original True IDE Mode. To configure the CompactFlash Storage Card in True IDE Mode, the 50-pin socket must be power cycled with the CompactFlash Storage Card inserted and -OE (output enable) asserted.

Function Code	-CS1	-CS0	A0-A2	-IORD	-IOWR	D15-D8	D7-D0
Invalid Mode	L	L	X	X	X	High Z	High Z
Standby Mode	H	H	X	X	X	High Z	High Z
Task File Write	H	L	1-7h	H	L	Don't Care	Data In
Task File Read	H	L	1-7h	L	H	High Z	Data Out
Data Register Write	H	L	0	H	L	Odd-Byte In	Even-Byte In
Data Register Read	H	L	0	L	H	Odd-Byte Out	Even-Byte Out
Control Register Write	L	H	6h	H	L	Don't Care	Control In
Alt Status Read	L	H	6h	L	H	High Z	Status Out
Drive Address ⁶	L	H	7h	L	H	High Z	Data Out

Figure 4: True IDE Mode I/O Function

1.5 True IDE Mode Addressing

When the CompactFlash Storage Card is configured in the True IDE Mode, the I/O decoding is as follows:

-CS1	-CS0	A2	A1	A0	-IORD=0	-IOWR=0	Note
1	0	0	0	0	RD Data	WR Data	8 or 16 bit ¹
1	0	0	0	1	Error Register	Features	8 bit
1	0	0	1	0	Sector Count	Sector Count	8 bit
1	0	0	1	1	Sector No.	Sector No.	8 bit
1	0	1	0	0	Cylinder Low	Cylinder Low	8 bit
1	0	1	0	1	Cylinder High	Cylinder High	8 bit
1	0	1	1	0	Select Card/Head	Select Card/Head	8 bit
1	0	1	1	1	Status	Command	8 bit
0	1	1	1	0	Alt Status	Device Control	8 bit

Figure 5: True IDE Mode I/O Decoding

1.6 Register Configuration

1) Data Register (Address - 1F0h[170h]; Offset 0,8,9)

Data Register True IDE Mode	-CS1	-CS0	A0	Offset	Data Bus
Word Data Register	1	0	0	0	D15-D0
Byte Data Register (Selected Using Set Features Command)	1	0	0	0	D7-D0

Figure 6: Data Register Access

2) Feature Register (Address - 1F1h[171h]; Offset 1, 0Dh Write Only)

3) Sector Count Register (Address - 1F2h[172h]; Offset 2)

4) Sector Number (LBA 7-0) Register (Address - 1F3h[173h]; Offset 3)

5) Cylinder Low (LBA 15-8) Register (Address - 1F4h[174h]; Offset 4)

6) Cylinder High (LBA 23-16) Register (Address - 1F5h[175h]; Offset 5)

7) Drive/Head (LBA 27-24) Register (Address - 1F6h[176h]; Offset 6)

D7	D6	D5	D4	D3	D2	D1	D0
1	LBA	1	DRV	HS3	HS2	HS1	HS0

Figure 7: Drive/Head Register

8) Status & Alternate Status Registers (Address - 1F7h[177h] & 3F6h[376h]; Offsets 7 & Eh)

D7	D6	D5	D4	D3	D2	D1	D0
BUSY	RDY	DWF	DSC	DRQ	CORR	0	ERR

Figure 8: Status & Alternate Status Register

9) Read Sector(s) - 20h or 21h

Bit ->	7	6	5	4	3	2	1	0
Command (7)	20h or 21h							
C/D/H (6)	1	LBA	1	Drive	Head (LBA 27-24)			
Cyl High (5)	Cylinder High (LBA 23-16)							
Cyl Low (4)	Cylinder Low (LBA 15-8)							
Sec Num (3)	Sector Number (LBA 7-0)							
Sec Cnt (2)	Sector Count							
Feature (1)	X							

Figure 9:Read Sector(s)

2 CF Card Reader Board

2.1 Block Diagram

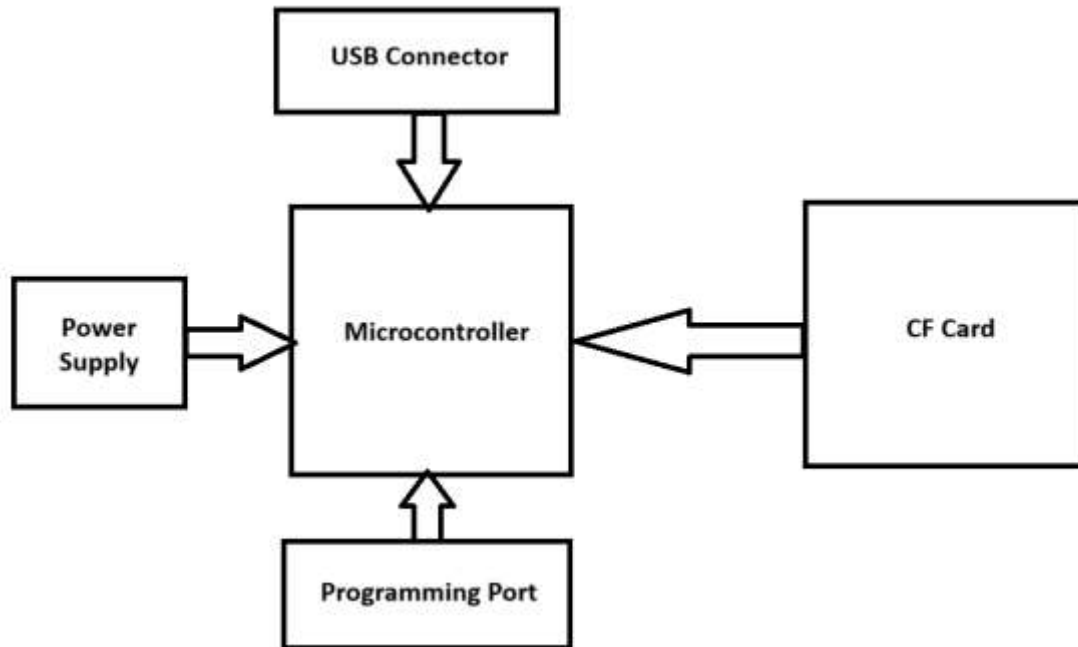


Figure 10: Block Diagram of CF Card Reader

2.2 Schematic Diagram

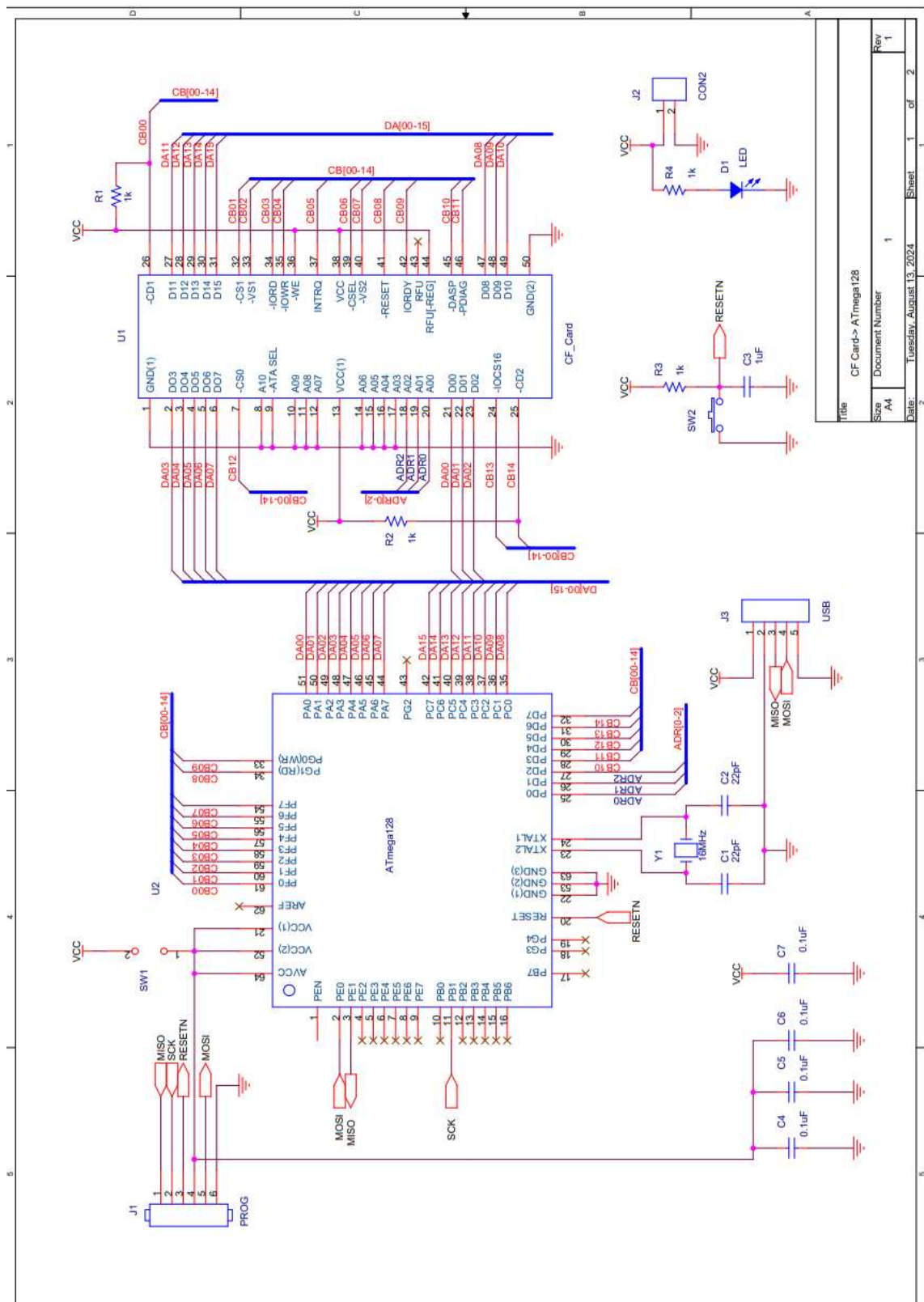


Figure 11: Schematic Diagram of CF Card Reader Board

2.3 PCB Design

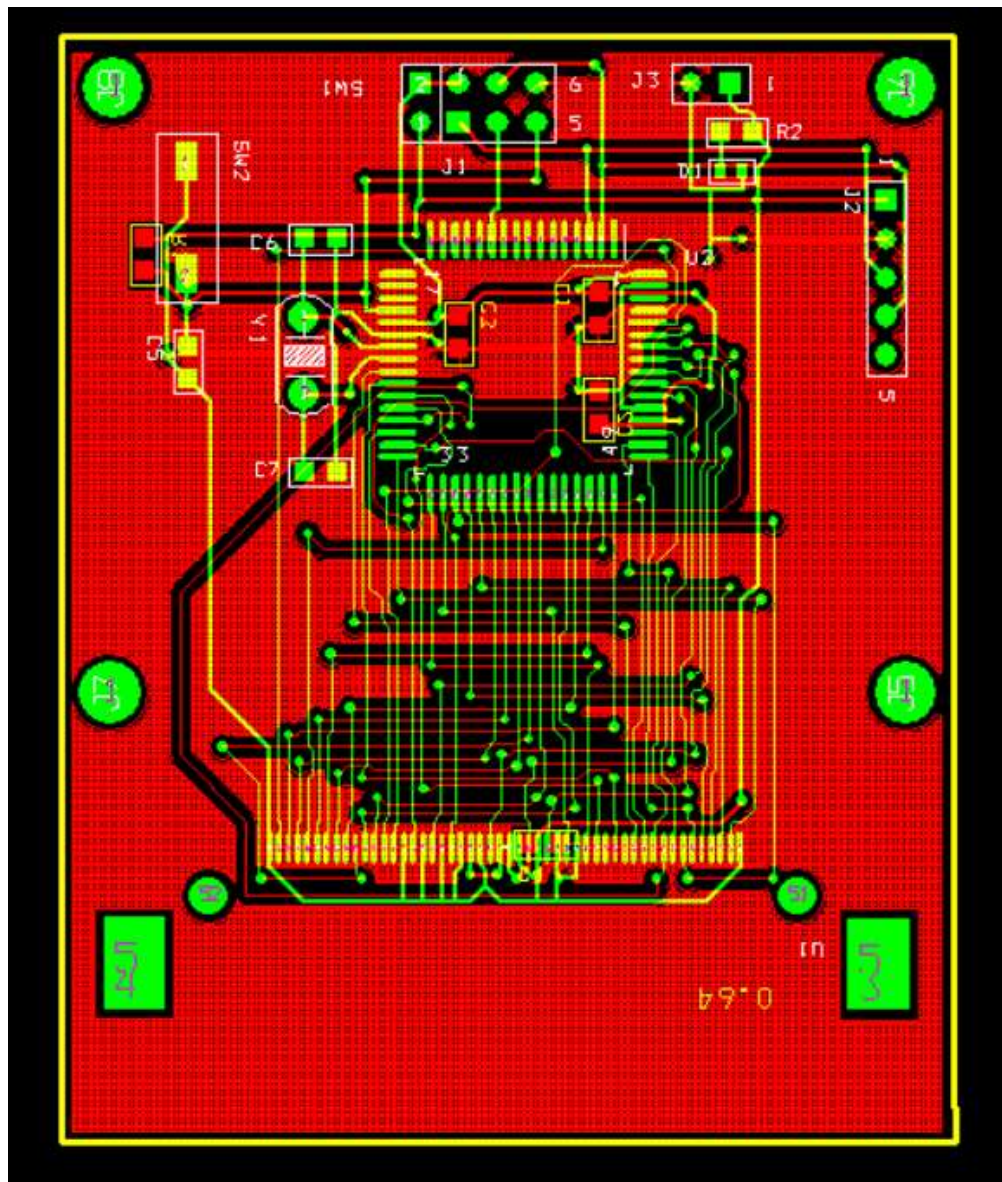


Figure 12: PCB Design of CF Card Reader Board

2.4 Printed Circuit Board



Figure 13: CF Card Reader Board with CF Card

3 FAT 32 File Structure

Structure



Figure 14: FAT 32 File System Structure

Master Boot Record

The drive's first sector is the Master Boot Record (LBA = 0). For any new projects, All IDE drives support accessing sectors using LBA. Also, all IDE drives use sectors that are 512 bytes

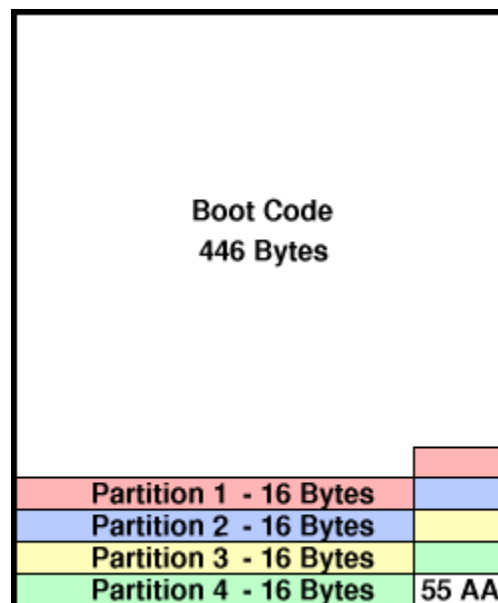


Figure 15: Master Boot Record

The first 446 bytes of the MBR are code that boots the computer. This is followed by a 64 byte partition table, and the last two bytes are always 0x55 and 0xAA. Check these last two bytes, as a simple "sanity check" that the MBR is ok.

Partition

The MBR can only represent four partitions. Each partition description is just 16 bytes, The fifth byte is a *Type Code* that tells what type of filesystem is supposed to be contained within the partition, and the ninth through twelfth bytes indicate the *LBA Begin* address where that partition begins on the disk.

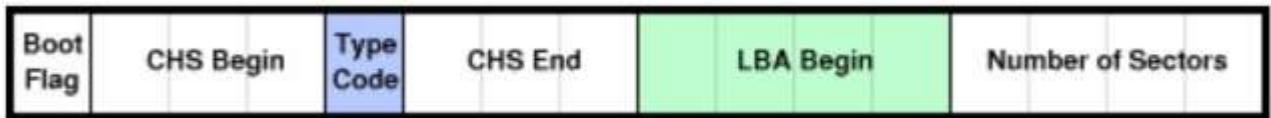


Figure 16: Partition Entry

Check the *Type Code* of each entry, looking for either 0x0B or 0x0C (the two that are used for FAT32), and then read the *LBA Begin* to learn where the FAT32 filesystem is located on the disk.

Volume ID

The first step to reading the FAT32 filesystem is the read its first sector, called the Volume ID. The Volume ID is read using the LBA Begin address found in the partition table. In this sector, contains data about the physical layout of the FAT32 filesystem.

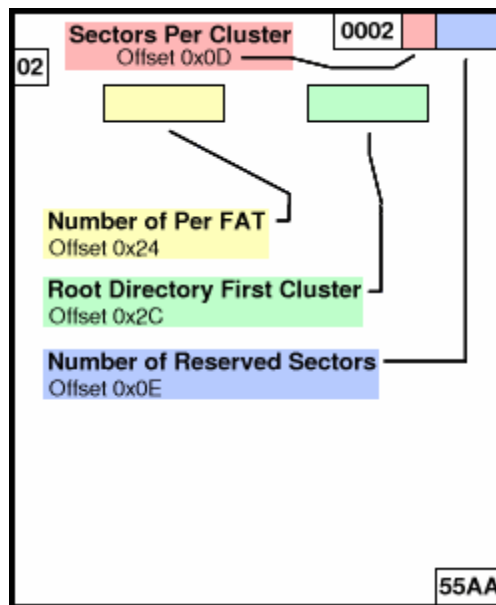


Figure 17: FAT32 Volume ID, critical fields

Field	Microsoft's Name	Offset	Size	Value
Bytes Per Sector	BPB_BytsPerSec	0x0B	16 Bits	Always 512 Bytes
Sectors Per Cluster	BPB_SecPerClus	0x0D	8 Bits	1,2,4,8,16,32,64,128
Number of Reserved Sectors	BPB_RsvdSecCnt	0x0E	16 Bits	Usually 0x20
Number of FATs	BPB_NumFATs	0x10	8 Bits	Always 2
Sectors Per FAT	BPB_FATSz32	0x24	32 Bits	Depends on disk size
Root Directory First Cluster	BPB_RootClus	0x2C	32 Bits	Usually 0x00000002
Signature	(none)	0x1FE	16 Bits	Always 0xAA55

Figure 18: Critical Fields of Volume ID and values

After checking the three fields to make sure the filesystem is using 512-byte sectors, 2 FATs, and has a correct signature

Important variables for Reading fat-32:

```
(unsigned long)fat_begin_lba = Partition_LBA_Begin + Number_of_Reserved_Sectors;  
  
(unsigned long)cluster_begin_lba = Partition_LBA_Begin +  
    Number_of_Reserved_Sectors + (Number_of_FATs * Sectors_Per_FAT);  
  
(unsigned char)sectors_per_cluster = BPB_SecPerClus;  
  
(unsigned long)root_dir_first_cluster = BPB_RootClus;
```

Figure 19: Important Variables for Reading Fat-32

Arrangement of the FAT32 Filesystem

The first sector is always the Volume ID, which is followed by some unused space called the reserved sectors. Following the reserved sectors are two copies of the FAT (File Allocation Table). The remainder of the filesystem is data arranged in "clusters", with perhaps a tiny bit of unused space after the last cluster.

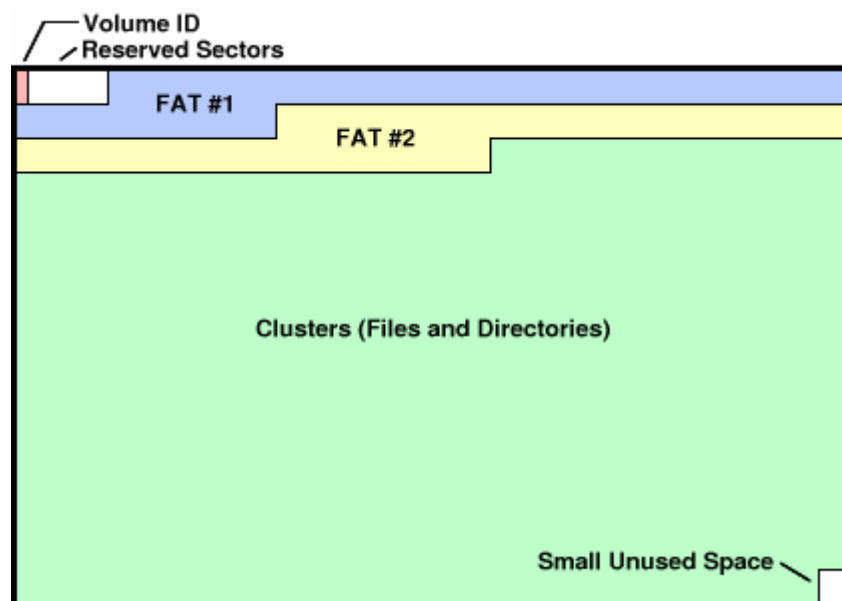


Figure 20: FAT32 Filesystem Overall Layout

The vast majority of the disk space is the clusters section, which is used to hold all the files and directories. The clusters begin their numbering at 2, so there is no cluster #0 or cluster #1. To access any particular cluster, Use this formula to turn the cluster number into the LBA address for the IDE drive

```
lba_addr = cluster_begin_lba + (cluster_number - 2) * sectors_per_cluster;
```

Figure 21: Formula to turn cluster number into the LBA address

When the beginning, Know the first cluster of the root directory. Reading the directory will reveal the names and first cluster locations of other files and subdirectories. A key point is that **directories only tell how to find the first cluster number of their files and subdirectories**. A directory only tells where the files begin. To access more than the first cluster, need to use the FAT. But first need to be able to find where those files start.

Directory data is structured into 32-byte records, which is convenient because each sector contains exactly 16 records, ensuring that no directory record crosses a sector boundary. There are four types of 32-byte directory records:

1. **Normal Record with Short Filename:** These records have a standard attribute.
2. **Long Filename Text:** These records have all four type bits set in the attribute field.
3. **Unused Record:** These are identified by a first byte of 0xE5.
4. **End of Directory:** These records start with a first byte of zero.

Unused directory records result from deleting files. When a file is deleted, its first byte is overwritten with 0xE5. These records can be reused when a new file is created. An end-of-directory record, which begins with zero, signals the conclusion of the directory. All other records will have a non-zero first byte, making it straightforward to identify the directory's end.

Records that do not start with 0xE5 or zero contain actual directory data, and their format is determined by checking the attribute byte. For our purposes, we will focus on normal directory records using the old 8.3 short filename format. In FAT32, all files and subdirectories have short names, even if a user assigns a longer name to a file. This allows access to all files without decoding long filename records, as long as the code simply ignores them.

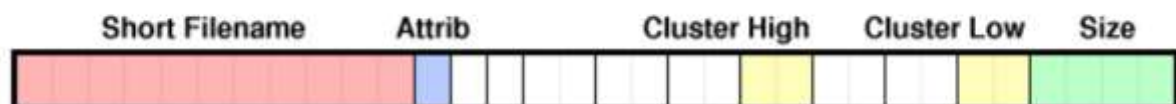


Figure 22: 32 Byte Directory Structure, Short Filename Format

Table 3: Field, Offset and Size of the 32 Byte Directory

Field	Offset	Size
Short Filename	0x00	11 Bytes
Attrib Byte	0x0B	8 Bits
First Cluster High	0x14	16 Bits
First Cluster Low	0x1A	16 Bits
File Size	0x1C	32 Bits

The Attrib byte has six bits defined, as shown in the table below. Most simple firmware will check the Attrib byte to determine if the 32 bytes are a normal record or long filename data, and to determine if it is a normal file or a subdirectory. Long filename records have all four of the least significant bits set. Normal files rarely have any of these four-bit sets.

Table 4: Attribute Byte Bit Functions in Directory Records

Attrib Bit	Function	LFN	Comment
0 (LSB)	Read Only	1	Should not allow writing
1	Hidden	1	Should not show in dir listing
2	System	1	File is an operating system
3	Volume ID	1	The filename is Volume ID
4	Directory	x	Is a subdirectory (32-byte records)
5	Archive	x	Has been changed since last backup
6	Unused	0	Should be zero
7 (MSB)	Unused	0	Should be zero

File Allocation Table - Following Cluster Chains

Directory entries indicate the location of the first cluster of each file or subdirectory on the disk. The first cluster of the root directory is determined from the volume ID sector. For small files and directories that fit within a single cluster, the File Allocation Table (FAT) indicates that there are no additional clusters. Access to clusters beyond the first requires the use of the FAT, which in FAT32 consists of 32-bit entries. FAT16 and FAT12 use 16-bit and 12-bit entries, respectively.

The FAT is a large array of 32-bit integers, where each position corresponds to a cluster number, and the stored value indicates the next cluster in the file. Each sector of the FAT contains 128 integers, facilitating the retrieval of the next cluster for a file. Bits 7-31 of the current cluster determine the sectors to read from the FAT, and bits 0-6 indicate which integer in that sector contains the next cluster number. If all bits are ones, it signifies the last cluster.

In the accompanying visual example, the root directory begins at cluster 2, as specified by the Volume ID. The number at position 2 in the FAT is 9, indicating that the next cluster is #9. Clusters A, B, and 11

contain the remainder of the root directory, with cluster 11 marked as the last. Generally, the end-of-directory marker (first byte is zero) is identified before reaching the last cluster.

For files, the FAT does not specify the first cluster; this information is found in the directory, while the FAT is used for subsequent clusters. The number of clusters allocated matches the file size. A portion of the last cluster may remain unused unless the file size is an exact multiple of the cluster size. Zero-length files do not have any clusters, and their directory cluster number is zero. Files that fit within a single cluster will have the "FFFFFFF" end-of-file marker at their FAT position. Files that fit within a

XXXXXXXX	XXXXXXXX	00000009	00000004
00000005	00000007	00000000	00000008
FFFFFFFF	0000000A	0000000B	00000011
0000000D	0000000E	FFFFFFFF	00000010
00000012	FFFFFFFF	00000013	00000014
00000015	00000016	FFFFFFFF	00000000
00000000	00000000	00000000	00000000
00000000	00000000	00000000	00000000
00000000	00000000	00000000	00000000
00000000	00000000	00000000	00000000
00000000	00000000	00000000	00000000
00000000	00000000	00000000	00000000
00000000	00000000	00000000	00000000
00000000	00000000	00000000	00000000
00000000	00000000	00000000	00000000
00000000	00000000	00000000	00000000

Root Directory:
2, 9, A, B, 11

File #1:
3, 4, 5, 7, 8

File #2:
C, D, E

File #3:
F, 10, 12, 13, 14, 15, 16

Figure 23: File Allocation Table and Following Cluster Chains

3.1 Master Boot Record

```
struct MBR_Structure{
    unsigned char bootStrapCode[446];
    unsigned char partitionEntry_1[16];
    unsigned char partitionEntry_2[16];
    unsigned char partitionEntry_3[16];
    unsigned char partitionEntry_4[16];
    uint16_t      BootSignature;
};
```

Figure 24: MBR Structure

```
struct partition_entry_Structure {
    unsigned char CurrentStateOfPartition;
    unsigned char BeginningOfPartitionHead;
    uint16_t      BeginningOfPartitionSectorCylinder;
    unsigned char TypeOfPartition;
    unsigned char EndOfPartitionHead;
    uint16_t      EndOfPartitionSectorCylinder;
    uint32_t      NumberOfSectorsBetweenMBR_FirstSectorPartition;
    uint32_t      NumberOfSectorsPartition;
};
```

Figure 25: Partition Entry Structure

Flash	EEPROM	Fuse Bits/Settings						
	00	01	02	03	04	05	06	07
000180								
000188	00	00	00	00	00	00	00	00
000190	00	00	00	00	00	00	00	00
000198	00	00	00	00	00	00	00	00
0001A0	00	00	00	00	00	00	00	00
0001A8	00	00	00	00	00	00	00	00
0001B0	00	00	00	00	00	00	00	00
0001B8	00	00	00	00	00	00	80	00
0001C0	03	00	0C	FA	FF	FB	FE	00
0001C8	00	00	72	39	BA	03	00	00
0001D0	00	00	00	00	00	00	00	00
0001D8	00	00	00	00	00	00	00	00
0001E0	00	00	00	00	00	00	00	00
0001E8	00	00	00	00	00	00	00	00
0001F0	00	00	00	00	00	00	00	00
0001F8	00	00	00	00	00	00	55	AA

Figure 26: Master Boot Record Layout

Table 5: Master Boot Record Data

Name	Colour	Offset	Size(Bytes)
BootStrapCode	No Colour	0x000000	446
partitionEntry_1	Green	0x0001BE	16
partitionEntry_2	Purple	0x0001CE	16
partitionEntry_3	Blue	0x0001DE	16
partitionEntry_4	Red	0x0001EE	16
BootSignature	Yellow	0x0001FE	2

0001A8	00	00	00	00	00	00	00	00
0001B0	00	00	00	00	00	00	00	00
0001B8	00	00	00	00	00	00	80	00
0001C0	03	00	0C	FA	FF	FB	FE	00
0001C8	00	00	72	39	BA	03	00	00
0001D0	00	00	00	00	00	00	00	00
0001D8	00	00	00	00	00	00	00	00

Figure 27: Partition_1 Entry

Table 6: Partition Entry_1 Data

Name	Colour	Offset	Size(Bytes)	Hex Value
Boot Flag	Green	0x0001BE	1	0x80
CHS Begin	Purple	0x0001BF	3	0x03
Type code	Blue	0x0001C2	1	0x0C
CHS END	Gray	0x0001C3	3	0xFBFFFA
LBA Begin	Red	0x0001C6	4	0xFE
Number of Sectors	Yellow	0x0001CA	4	0x03BA3972

3.2 Volume ID of the first partition.

```
struct BS_Structure{
    unsigned char jumpBoot[3];
    unsigned char OEMName[8];
    unsigned int bytesPerSector;
    unsigned char sectorPerCluster;
    unsigned int reservedSectorCount;
    unsigned char numberOfFATs;
    unsigned int rootEntryCount;
    unsigned int totalSectors_F16;
    unsigned char mediaType;
    unsigned int FATsize_F16;
    unsigned int sectorsPerTrack;
    unsigned int numberOfHeads;
    unsigned long hiddenSectors;
    unsigned long totalSectors_F32;
    unsigned long FATsize_F32;
    unsigned int extFlags;
    unsigned int Version_of_fat32_driver;
    unsigned long ClusterNumberOfRoot;
    unsigned int sector_number_FSinfo;
    unsigned int sector_no_BackupBoot;
    unsigned char reserved[12];
    unsigned char driveNumber;
    unsigned char reserved1;
    unsigned char bootSignature;
    unsigned long volumeID;
    unsigned char volumeLabel[11];
    unsigned char FAT_name[8];
    unsigned char bootData[420];
    unsigned int bootEndSignature;
};
```

Figure 28: Boot Sector Structure

Flash	EEPROM	Fuse Bits/Settings						
	00	01	02	03	04	05	06	07
000000	EB	00	90	20	20	20	20	20
000008	20	20	20	00	02	40	20	00
000010	02	00	00	00	00	F8	00	00
000018	3F	00	FF	00	FE	00	00	00
000020	72	3D	BA	03	D1	1D	00	00
000028	00	00	00	00	02	00	00	00
000030	01	00	06	00	00	00	00	00
000038	00	00	00	00	00	00	00	00
000040	80	00	29	75	F4	E2	48	4E
000048	4F	20	4E	41	4D	45	20	20
000050	20	20	46	41	54	33	32	20
000058	20	20	00	00	00	00	00	00
000060	00	00	00	00	00	00	00	00
000068	00	00	00	00	00	00	00	00
000070	00	00	00	00	00	00	00	00

Figure 29: Volume ID (Boot Sector)

	00	01	02	03	04	05	06	07
000000	EB	00	90	20	20	20	20	20
000008	20	20	20	00	02	40	20	00
000010	02	00	00	00	00	F8	00	00
000018	3F	00	FF	00	FE	00	00	00
000020	72	39	BA	03	D1	1D	00	00
000028	00	00	00	00	02	00	00	00
000030	01	00	06	00	00	00	00	00
000038	00	00	00	00	00	00	00	00

Figure 30: Fat 32 Volume ID, Critical Fields

Table 7: Fat 32 Volume ID and Critical Fields Data

Field	Colour	Offset	Size (Bytes)	Hex Value	Dec Value
Bytes Per Sector	Green	0x0B	2	0x200	512 Bytes
Sectors Per Cluster	Red	0x0D	1	0x40	64
Number of Reserved Sectors	Purple	0x0E	2	0x20	32
Number of FATs	Blue	0x10	1	0x02	2
Sectors Per FAT	Black	0x24	4	0x1D D1	7633
Root Directory First Cluster	Yellow	0x2C	4	0x02	2

3.3 Necessary Calculations

1) Fat Begin Address

$$\text{Fat Begin Address} = \text{Partition Begin Address} + \text{Reserved Sector Count}$$

$$\begin{aligned}\text{Partition Begin Address} &= 0xFE \\ \text{Reserved Sector Count} &= 0x20 \\ \text{Fat Begin Address} &= 0xFE + 0x20 \\ &= \underline{0x011E} \text{ (Sector no = 286)}\end{aligned}$$

2) Cluster Begin Address

$$\text{Cluster Begin Address} = \text{Fat Begin Address} + (\text{Number of FATs} \times \text{FATsize_F32})$$

$$\begin{aligned}\text{Fat Begin Address} &= 0x011E \\ \text{Number of FATs} &= 0x02 \\ \text{FATsize_F32} &= 0x1DD1 \\ \text{Cluster Begin Address} &= 0x011E + (0x02 \times 0x1DD1) \\ &= \underline{0x3CC0} \text{ (Sector no = 15552)}\end{aligned}$$

3) Root Dir First Cluster Address

$$\text{Root Dir First Cluster Address} = \text{Cluster Begin Address} + [(\text{Root Dir First Cluster} - 2) \times (\text{Sec Per Cluster})]$$

$$\begin{aligned}\text{Cluster Begin Address} &= 0x3CC0 \\ \text{Root Dir First Cluster} &= 0x02 \\ \text{Sectors Per Cluster} &= 0x40 \\ \text{Root Dir First Cluster Address} &= 0x3CC0 + [(0x02 - 0x02) \times 0x40] \\ &= \underline{0x3CC0} \text{ (Sector no = 15552)}\end{aligned}$$

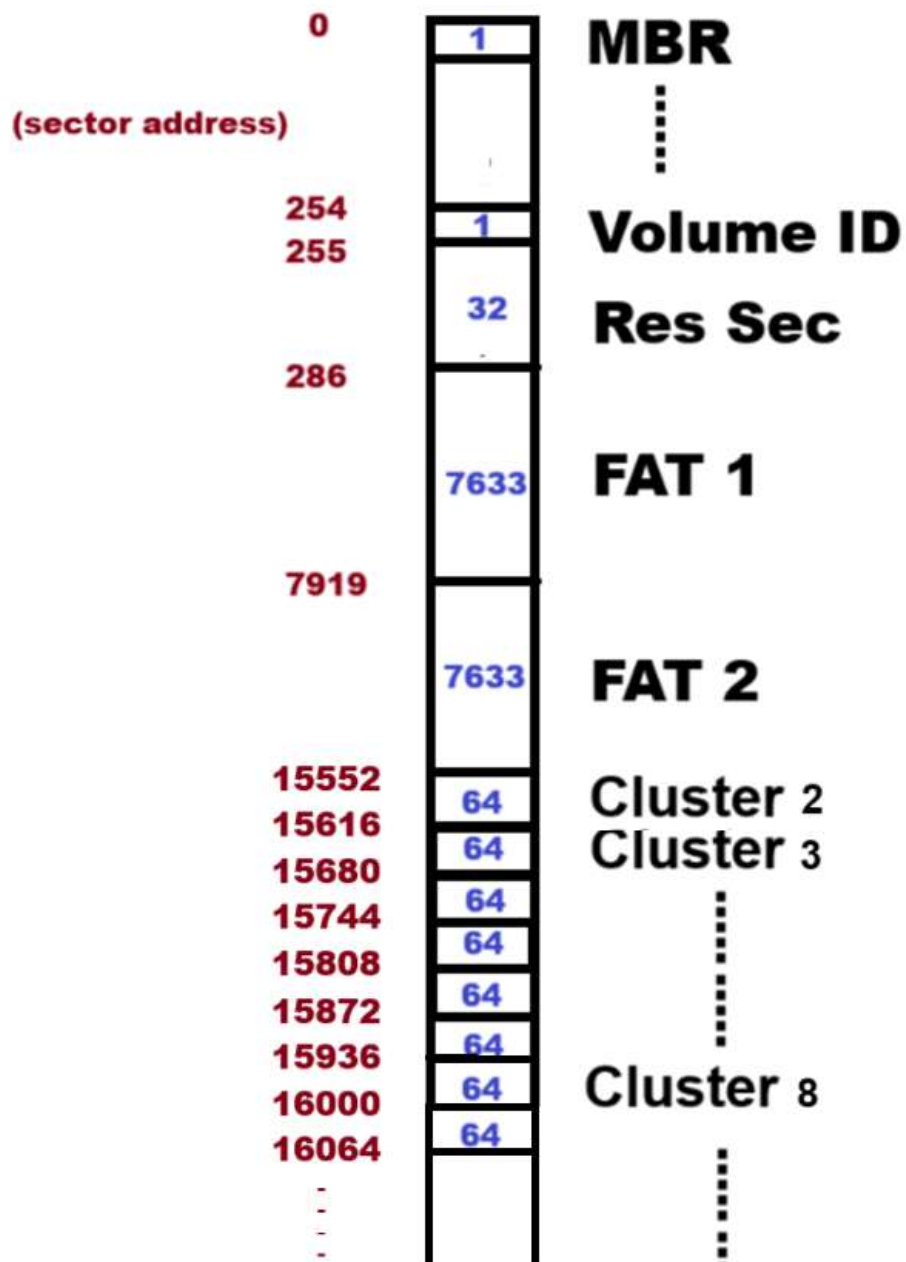


Figure 31: Sector Locations

The starting cluster of the Root Directory is Cluster Two (Starts from Sector 15552)

3.4 How to locate a file in the Root Directory

Offset(d)	00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	Decoded text
00007962768	6F	00	6F	00	73	00	74	00	50	00	00	00	65	00	72	00	o.o.s.t.P...e.r.
00007962784	E5	45	41	44	59	42	7E	31	54	4D	50	20	00	30	D4	42	AEADYB~1TMP .00B
00007962800	01	59	01	59	00	00	D5	42	01	59	05	00	00	00	80	00	.Y.Y..0B.Y....e.
00007962816	E5	43	00	46	00	2D	00	4D	00	43	00	0F	00	98	2E	00	AC.F.-.M.C...".
00007962832	61	00	74	00	73	00	6C	00	6E	00	00	00	00	00	FF	FF	a.t.s.i.n....yy
00007962848	E5	46	2D	4D	43	7E	31	20	41	54	53	20	00	83	4A	7F	AF-MC~1 ATS .fJ.
00007962864	0B	59	0E	59	00	00	37	AF	08	59	06	00	C5	03	00	00	.Y.Y..7.Y..A...
00007962880	E5	41	54	5F	33	32	20	20	48	20	20	20	10	34	69	81	AA_T 32 H .4i.
00007962896	0B	59	0E	59	00	00	C6	42	07	59	07	00	63	03	00	00	.Y.Y..EB.Y..c...
00007962912	E5	2E	00	76	00	73	00	00	00	FF	FF	0F	00	EC	FF	FF	A..v.s...yy..iyy
00007962928	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	00	00	FF	FF	FF	yyyyyyyyyy..yyyy
00007962944	E5	53	34	44	41	35	7E	31	20	20	20	12	00	7B	73	81	AS4DA5~1 ..(s.
00007962960	0B	59	0B	59	00	00	74	81	0B	59	08	00	00	80	00	00	.Y.Y..t..Y..e..
00007962976	E5	6D	00	65	00	6E	00	74	00	2E	00	0F	00	9F	74	00	Am.e.n.t....Yt.
00007962992	78	00	74	00	00	00	FF	FF	FF	FF	00	00	FF	FF	FF	FF	x.t...yyyy..yyyy
00007963008	E5	4E	00	65	00	77	00	20	00	54	00	0F	00	9F	65	00	AN.e.w. .T...Ye.
00007963024	78	00	74	00	20	00	44	00	6F	00	00	00	63	00	75	00	x.t. .D.o...c.u.
00007963040	E5	45	57	54	45	58	7E	31	54	58	54	20	00	53	5C	55	AEWTEX~1TXT .S\U
00007963056	0E	59	0E	59	00	00	5D	55	0E	59	00	00	00	00	00	00	.Y.Y..]U.Y.....
00007963072	41	42	43	20	20	20	20	20	54	58	54	20	18	53	5C	55	ABC TXT .S\U
00007963088	0E	59	0E	59	00	00	6C	55	0E	59	08	00	10	00	00	00	.Y.Y..lU.Y.....
00007963104	E5	6D	00	65	00	6E	00	74	00	2E	00	0F	00	9F	74	00	Am.e.n.t....Yt.
00007963120	78	00	74	00	00	00	FF	FF	FF	FF	00	00	FF	FF	FF	FF	x.t...yyyy..yyyy
00007963136	E5	4E	00	65	00	77	00	20	00	54	00	0F	00	9F	65	00	AN.e.w. .T...Ye.
00007963152	78	00	74	00	20	00	44	00	6F	00	00	00	63	00	75	00	x.t. .D.o...c.u.
00007963168	E5	45	57	54	45	58	7E	31	54	58	54	20	00	C1	17	59	AEWTEX~1TXT .A.Y
00007963184	0E	59	0E	59	00	00	18	59	0E	59	00	00	00	00	00	00	.Y.Y...Y.Y.....
00007963200	31	32	33	20	20	20	20	54	58	54	20	10	C1	17	59		123 TXT .A.Y
00007963216	0E	59	0E	59	00	00	24	59	0E	59	09	00	0D	00	00	00	.Y.Y..SY.Y.....
00007963232	E5	6D	00	65	00	6E	00	74	00	2E	00	0F	00	9F	74	00	Am.e.n.t....Yt.
00007963248	78	00	74	00	00	00	FF	FF	FF	FF	00	00	FF	FF	FF	FF	x.t...yyyy..yyyy
00007963264	E5	4E	00	65	00	77	00	20	00	54	00	0F	00	9F	65	00	AN.e.w. .T...Ye.
00007963280	78	00	74	00	20	00	44	00	6F	00	00	00	63	00	75	00	x.t. .D.o...c.u.

Figure 32: Directory Entry of the File ABC

41 42 43 20 20 20 20 20 54 58 54 20 18 53 5C 55	ABC	TXT	.S\U
0E 59 0E 59 00 00 6C 55 0E 59 08 00 10 00 00 00			.Y.Y..lU.Y.....

Figure 33: 32-byte Directory Layout

Table 8: Data of the Directory Entry of the File ABC

Field	Offset	Size	Color	Value
Short Filename	0x00	11 Bytes	Red	ABC TXT
Attrib Byte	0x0B	8 Bits	Purple	32
First Cluster High	0x14	16 Bits	Green	0x0000
First Cluster Low	0x1A	16 Bits	Blue	0x0008
File Size	0x1C	32 Bits	Yellow	0x00000010

High order value: 00 00 -> little-endian to big-endian -> 00 00

Low-order value: 08 00 -> little-endian to big-endian -> 00 08

Cluster number = 00 00 00 08 = 8 (decimal)

Therefore, the first cluster where the file contents can be found is cluster 8. Cluster 8 starts from Sector 15936.

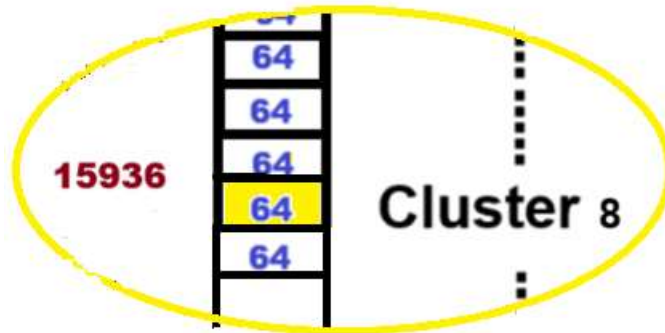


Figure 34: Location of Cluster 8

Offset (h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text	
0007C8000	74	68	69	73	61	72	61	20	64	68	61	74	68	69	79	61	thisara dhathiya	Sector 15,936
0007C8010	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C8020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C8030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C8040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C8050	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C8060	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C8070	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C8080	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C8090	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C80A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C80B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C80C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C80D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C80E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C80F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C8100	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C8110	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C8120	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C8130	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C8140	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C8150	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C8160	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C8170	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C8180	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C8190	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C81A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C81B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C81C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C81D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C81E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C81F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C8200	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		Sector 15,937
0007C8210	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0007C8220	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
.....	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..		

Figure 35: First Sector of the File ABC

3.5 How to locate a Sub-Directory (folder) in the Root Directory

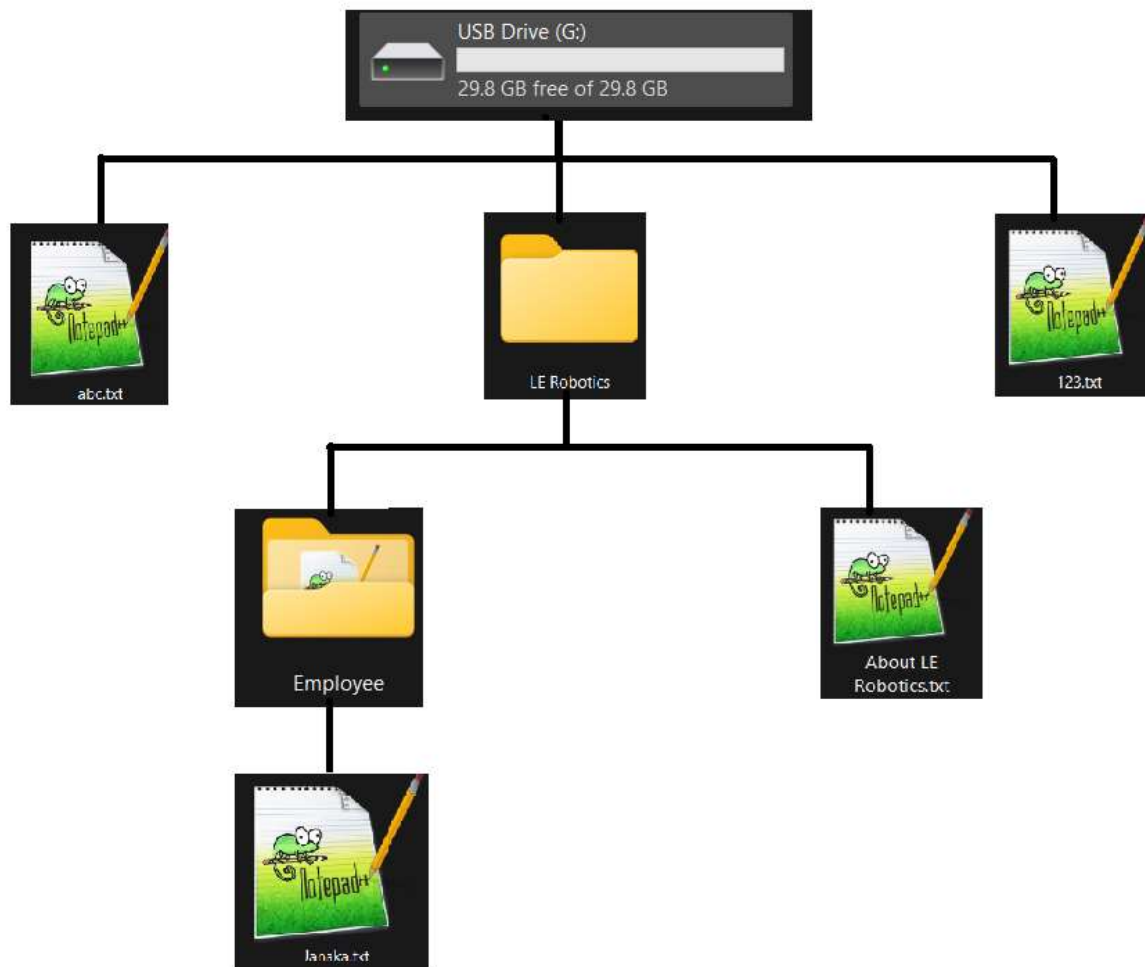


Figure 36: Files and Folders in CF Card

```

Offset(h) 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F Decoded text
0007984A0 E5 32 00 33 00 34 00 35 00 36 00 0F 00 FA 37 00 Å2.3.4.5.6...å7.
0007984B0 38 00 39 00 61 00 62 00 63 00 00 00 64 00 65 00 8.9.a.b.c...d.e.
0007984C0 E5 68 00 69 00 6C 00 61 00 6B 00 0F 00 FA 61 00 Åh.i.l.a.k...åa.
0007984D0 72 00 61 00 74 00 68 00 6E 00 00 00 61 00 31 00 r.a.t.h.n...a.l.
0007984E0 E5 73 00 61 00 6E 00 61 00 74 00 0F 00 FA 68 00 Ås.a.n.a.t...åh.
0007984F0 6B 00 75 00 6D 00 61 00 72 00 00 00 61 00 74 00 k.u.m.a.r...a.t.
000798500 E5 41 4E 41 54 48 7E 32 54 58 54 20 00 35 56 81 ÅANATH-2TXT .5V.
000798510 0E 59 0E 59 00 00 29 81 0E 59 07 00 55 00 00 00 .Y.Y..Y..U...
000798520 E5 66 00 67 00 68 00 69 00 6A 00 0F 00 FA 2E 00 Åf.g.h.i.j...å..
000798530 74 00 78 00 74 00 00 00 FF FF 00 00 FF FF FF FF t.x.t...yy...yyy
000798540 E5 32 00 33 00 34 00 35 00 36 00 0F 00 FA 37 00 Å2.3.4.5.6...å7.
000798550 38 00 39 00 61 00 62 00 63 00 00 00 64 00 65 00 8.9.a.b.c...d.e.
000798560 E5 68 00 69 00 6C 00 61 00 6B 00 0F 00 FA 61 00 Åh.i.l.a.k...åa.
000798570 72 00 61 00 74 00 68 00 6E 00 00 00 61 00 31 00 r.a.t.h.n...a.l.
000798580 E5 73 00 61 00 6E 00 61 00 74 00 0F 00 FA 68 00 Ås.a.n.a.t...åh.
000798590 6B 00 75 00 6D 00 61 00 72 00 00 00 61 00 74 00 k.u.m.a.r...a.t.
0007985A0 E5 41 4E 41 54 48 7E 32 54 58 54 20 00 35 56 81 ÅANATH-2TXT .5V.
0007985B0 0E 59 0E 59 00 00 69 81 0E 59 07 00 7B 00 00 00 .Y.Y..i..Y..f...
0007985C0 41 4C 00 45 00 20 00 52 00 6F 00 0F 00 67 62 00 AL.E. .R.o...gb.
0007985D0 6F 00 74 00 69 00 63 00 73 00 00 00 00 00 FF FF o.t.i.c.s...yy
0007985E0 4C 45 52 4F 42 4F 7E 31 20 20 20 10 00 0A 27 44 LEROBO~1 ...'D
0007985F0 0F 59 0F 59 00 00 28 44 0F 59 0C 00 00 00 00 00 .Y.Y..(D.Y.....
000798600 E5 29 00 00 00 FF FF FF FF FF FF FF FF FF FF a)...YYYYYY...YYY
000798610 FF FF FF FF FF FF FF FF FF FF FF FF FF FF YYYYYYYYYY...YYY
000798620 E5 4E 00 65 00 77 00 20 00 66 00 0F 00 FD 6F 00 ÅN.e.w. .f...yo.
000798630 6C 00 64 00 65 00 72 00 20 00 00 00 28 00 32 00 l.d.e.r. ...(.2.
000798640 E5 45 57 46 4F 4C 7E 32 20 20 20 10 00 06 2B 44 ÅEWFOI-2 ...+D
000798650 0F 59 0F 59 00 00 2C 44 0F 59 0D 00 00 80 00 00 .Y.Y..D.Y...e..
000798660 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000798670 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000798680 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

Figure 37: Directory Entry of the Folder "LE Robotics" in the Root Directory

```

0007985E0 4C 45 52 4F 42 4F 7E 31 20 20 20 10 00 0A 27 44 LEROBO~1 ...'D
0007985F0 0F 59 0F 59 00 00 28 44 0F 59 0C 00 00 00 00 00 .Y.Y..(D.Y.....

```

Figure 38: Layout of the Directory Entry of the Folder "LE Robotics"

Table 9: Data of the Directory Entry of the Folder "LE Robotics"

Field	Offset	Size	Color	Value
Short Filename	0x00	11 Bytes	Black	LEROBO~1
Attrib Byte	0x0B	8 Bits	Red	16
First Cluster High	0x14	16 Bits	Purple	0x0000
First Cluster Low	0x1A	16 Bits	Green	0x000C
File Size	0x1C	32 Bits	Yellow	0x00000000

High order value: 00 00 -> little-endian to big-endian -> 00 00

Low-order value: 0C 00 -> little-endian to big-endian -> 00 0C

Cluster number = 00 00 00 0C = 12 (decimal)

Therefore, the first cluster where the file contents can be found is cluster 12. Cluster 12 starts from Sector 16192

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
0007E8000	2E	20	20	20	20	20	20	20	20	20	10	00	0A	27	44		... 'D
0007E8010	0F	59	0F	59	00	00	28	44	0F	59	0C	00	00	00	00	00	.Y.Y..(D.Y.....
0007E8020	2E	2E	20	20	20	20	20	20	20	20	10	00	0A	27	44		... 'D
0007E8030	0F	59	0F	59	00	00	28	44	0F	59	00	00	00	00	00	00	.Y.Y..(D.Y.....
0007E8040	E5	49	00	6E	00	74	00	65	00	72	00	0F	00	48	6E	00	âI.n.t.e.r...Hn.
0007E8050	00	00	FF	FF	FF	FF	FF	FF	FF	FF	00	00	FF	FF	FF	FF	..YYYYYYYY..YYYY
0007E8060	E5	4E	54	45	52	4E	20	20	20	20	20	10	00	0C	42	44	âNTERN ...BD
0007E8070	0F	59	0F	59	00	00	43	44	0F	59	0D	00	00	80	00	00	.Y.Y..CD.Y...€..
0007E8080	E5	4E	00	65	00	77	00	20	00	66	00	0F	00	DD	6F	00	âN.e.w. .f...Yo.
0007E8090	6C	00	64	00	65	00	72	00	00	00	00	00	FF	FF	FF	FF	l.d.e.r....YYYY
0007E80A0	E5	45	57	46	4F	4C	7E	31	20	20	20	10	00	88	4C	44	âEWFOl-1 .. 'LD
0007E80B0	0F	59	0F	59	00	00	4D	44	0F	59	0E	00	00	80	00	00	.Y.Y..MD.Y...€..
0007E80C0	41	45	00	6D	00	70	00	6C	00	6F	00	0F	00	F8	79	00	âE.m.p.l.o...ey.
0007E80D0	65	00	65	00	00	00	FF	FF	FF	FF	00	00	FF	FF	FF	FF	e.e...YYYY..YYYY
0007E80E0	45	4D	50	4C	4F	59	45	45	20	20	20	10	00	12	51	44	EMPLOYEE ...QD
0007E80F0	0F	59	0F	59	00	00	52	44	0F	59	0E	00	00	00	00	00	.Y.Y..RD.Y.....
0007E8100	42	74	00	69	00	63	00	73	00	2E	00	0F	00	AE	74	00	Bt.i.c.s.....St.
0007E8110	78	00	74	00	00	00	FF	FF	FF	FF	00	00	FF	FF	FF	FF	x.t...YYYY..YYYY
0007E8120	01	41	00	62	00	6F	00	75	00	74	00	0F	00	AE	20	00	.A.b.o.u.t...@ .
0007E8130	4C	00	45	00	20	00	52	00	6F	00	00	00	62	00	6F	00	L.E. .R.o...b.o.
0007E8140	41	42	4F	55	54	4C	7E	31	54	58	54	20	00	24	5B	44	ABOUTL-1TXT .S[D
0007E8150	0F	59	0F	59	00	00	2B	46	0F	59	0D	00	A9	02	00	00	.Y.Y..+F.Y..@...
0007E8160	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0007E8170	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0007E8180	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0007E8190	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0007E81A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0007E81B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0007E81C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0007E81D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0007E81E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0007E81F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

Figure 39: Sub Directory of the Folder LE Robotics

3.6 How to locate a file in the Sub-Directory in the Root Directory

```

Offset(h) 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F Decoded text
0007E8000 2E 20 20 20 20 20 20 20 20 20 20 10 00 0A 27 44 . ...'D Sector 16,192
0007E8010 0F 59 0F 59 00 00 28 44 0F 59 0C 00 00 00 00 00 .Y.Y..(D.Y.....
0007E8020 2E 2E 20 20 20 20 20 20 20 20 20 10 00 0A 27 44 .. ...'D
0007E8030 0F 59 0F 59 00 00 28 44 0F 59 0C 00 00 00 00 00 .Y.Y..(D.Y.....
0007E8040 E5 49 00 6E 00 74 00 65 00 72 00 0F 00 48 6E 00 ÅI.n.t.e.r...Hn.
0007E8050 00 00 FF FF FF FF FF FF FF 00 00 FF FF FF FF ..YYYYYYYY..YYYY
0007E8060 E5 4E 54 45 52 4E 20 20 20 20 20 10 00 0C 42 44 ÅNTERN ...BD
0007E8070 0F 59 0F 59 00 00 43 44 0F 59 0D 00 00 80 00 00 .Y.Y..CD.Y...e..
0007E8080 E5 4E 00 65 00 77 00 20 00 66 00 0F 00 DD 6F 00 ÅN.e.w. .f...Yo.
0007E8090 6C 00 64 00 65 00 72 00 00 00 00 00 FF FF FF FF l.d.e.r.....YYYY
0007E80A0 E5 45 57 46 4F 4C 7E 31 20 20 20 10 00 88 4C 44 ÅEWFOl~l ...LD
0007E80B0 0F 59 0F 59 00 00 4D 44 0F 59 0E 00 00 80 00 00 .Y.Y..MD.Y...e..
0007E80C0 41 45 00 6D 00 70 00 6C 00 6F 00 0F 00 F8 79 00 AE.m.p.l.o...ey.
0007E80D0 65 00 65 00 00 00 FF FF FF FF 00 00 FF FF FF FF e.e...YYYY..YYYY
0007E80E0 45 4D 50 4C 4F 59 45 45 20 20 20 10 00 12 51 44 EMPLOYEE ...QD
0007E80F0 0F 59 0F 59 00 00 52 44 0F 59 0E 00 00 00 00 00 .Y.Y..RD.Y.....
0007E8100 42 74 00 69 00 63 00 73 00 2E 00 0F 00 AE 74 00 Bt.i.c.s.....@t.
0007E8110 78 00 74 00 00 00 FF FF FF FF 00 00 FF FF FF FF x.t...YYYY..YYYY
0007E8120 01 41 00 62 00 6F 00 75 00 74 00 0F 00 AE 20 00 .A.b.o.u.t...@.
0007E8130 4C 00 45 00 20 00 52 00 6F 00 00 00 62 00 6F 00 L.E. .R.e...b.e.
0007E8140 41 42 4F 55 54 4C 7E 31 54 58 54 20 00 24 5B 44 ABOUTL~1TXT .${D
0007E8150 0F 59 0F 59 00 00 2B 46 0F 59 0D 00 A9 02 00 00 .Y.Y..+F.Y..@...
0007E8160 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0007E8170 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0007E8180 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0007E8190 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0007E81A0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0007E81B0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

Figure 40: Sub Directory Entry of the File “About LE Robotics”

```

0007E8140 41 42 4F 55 54 4C 7E 31 54 58 54 20 00 24 5B 44 ABOUTL~1TXT .${D
0007E8150 0F 59 0F 59 00 00 2B 46 0F 59 0D 00 A9 02 00 00 .Y.Y..+F.Y..@...

```

Figure 41: Layout of the Directory Entry of the File “About LE Robotics”

Table 10: Data of the Directory Entry of the Folder "LE Robotics"

Field	Offset	Size	Color	Value
Short Filename	0x00	11 Bytes	Black	ABOUTL~1TXT
Attrib Byte	0x0B	8 Bits	Red	32
First Cluster High	0x14	16 Bits	Purple	0x0000
First Cluster Low	0x1A	16 Bits	Green	0x000D
File Size	0x1C	32 Bits	Yellow	681 bytes

High order value: 00 00 -> little-endian to big-endian -> 00 00

Low-order value: 0C 00 -> little-endian to big-endian -> 00 0D

Cluster number = 00 00 00 0D = 13 (decimal)

Therefore, the first cluster where the file contents can be found is cluster 13. Cluster 13 starts from Sector 16256,

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text	
0007F0000	57	65	20	61	72	65	20	74	68	65	20	66	69	72	73	74	We are the first	Sector 16,256
0007F0010	20	69	6E	20	74	68	65	20	53	72	69	20	4C	61	6E	6B	in the Sri Lank	
0007F0020	61	6E	20	6D	61	72	6B	65	74	20	74	6F	20	6F	66	66	an market to off	
0007F0030	65	72	20	61	20	66	75	6C	6C	79	20	63	75	73	74	6F	er a fully custo	
0007F0040	6D	69	73	61	62	6C	65	20	72	6F	62	6F	74	69	63	73	misable robotics	
0007F0050	20	73	6F	6C	75	74	69	6F	6E	73	20	6D	61	64	65	20	solutions made	
0007F0060	69	6E	20	53	72	69	20	4C	61	6E	6B	61	20	66	6F	72	in Sri Lanka for	
0007F0070	20	79	6F	75	72	20	61	75	74	6F	6D	61	74	69	6F	6E	your automation	
0007F0080	20	6E	65	65	64	73	20	66	6F	72	20	61	6E	20	61	66	needs for an af	
0007F0090	66	6F	72	64	61	62	6C	65	20	70	72	69	63	65	20	77	fordable price w	
0007F00A0	69	74	68	20	74	68	65	20	65	78	70	65	72	74	69	73	ith the expertis	
0007F00B0	65	20	62	61	73	65	64	20	69	6E	20	53	72	69	20	4C	e based in Sri L	
0007F00C0	61	6E	6B	61	2E	0D	0A	57	69	74	68	20	74	68	65	20	anka...With the	
0007F00D0	6C	6F	63	61	6C	6C	79	20	64	65	76	65	6C	6F	70	65	locally develope	
0007F00E0	64	20	65	6C	65	63	74	72	6F	6E	69	63	20	63	6F	6E	d electronic con	
0007F00F0	74	72	6F	6C	20	73	79	73	74	65	6D	73	20	61	6E	64	trol systems and	
0007F0100	20	6C	6F	63	61	6C	6C	79	20	64	65	73	69	67	6E	65	locally designe	
0007F0110	64	20	61	6E	64	20	6D	61	6E	75	66	61	63	74	75	72	d and manufactur	
0007F0120	74	6F	6D	69	73	61	74	69	6F	6E	20	66	6F	72	20	79	ed mechanical sy	
0007F0130	73	74	65	6D	73	2C	20	77	65	20	63	61	6E	20	61	73	stems, we can as	
0007F0140	73	75	72	65	20	79	6F	75	20	74	6F	20	70	72	6F	76	sure you to prov	
0007F0150	69	64	65	20	74	68	65	20	62	65	73	74	20	63	75	73	ide the best cus	
0007F0160	74	6F	6D	69	73	61	74	69	6F	6E	20	66	6F	72	20	79	tomisation for y	
0007F0170	6F	75	72	20	73	70	65	63	69	66	69	63	20	6E	65	65	our specific nee	
0007F0180	64	2E	0D	0A	57	6F	72	6C	64	E2	80	93	43	6C	61	73	d...World"Clas	
0007F0190	73	20	71	75	61	6C	69	74	79	20	67	65	61	72	20	62	s quality gear b	
0007F01A0	6F	78	65	73	2C	20	6D	6F	74	6F	72	73	20	61	6E	64	oxes, motors and	
0007F01B0	20	65	6C	65	63	74	72	6F	6E	69	63	20	63	6F	6D	70	electronic comp	
0007F01C0	6F	6E	65	6E	74	73	20	77	69	6C	6C	20	61	73	73	75	onents will assu	
0007F01D0	72	65	20	74	68	65	20	71	75	61	6C	69	74	79	20	61	re the quality a	
0007F01E0	6E	64	20	72	65	6C	69	61	62	69	6C	69	74	79	20	6F	nd reliability o	
0007F01F0	66	20	6F	75	72	20	70	72	6F	64	75	63	74	73	2E	0D	f our products..	
0007F0200	0A	42	65	69	6E	67	20	61	20	6C	6F	63	61	6C	6C	79	.Being a locally	Sector 16,257
0007F0210	20	62	61	73	65	64	20	73	6F	6C	75	74	69	6F	6E	20	based solution	
0007F0220	70	72	6F	76	69	64	65	72	2C	20	77	65	20	61	69	6D	provider, we aim	
0007F0230	20	74	6F	20	70	72	6F	76	69	64	65	20	61	6E	20	65	to provide an e	
0007F0240	61	73	69	6C	79	20	61	70	70	72	6F	61	63	68	61	62	asily approachab	
0007F0250	6C	65	20	61	6E	64	20	66	6C	65	78	69	62	6C	65	20	le and flexible	
0007F0260	73	65	72	76	69	63	65	20	61	74	20	61	6E	20	61	66	service at an af	
0007F0270	66	6F	72	64	61	62	6C	65	20	70	72	69	63	65	20	75	fordable price u	
0007F0280	6E	6D	61	74	63	68	65	64	20	77	69	74	68	20	66	6F	nmatched with fo	

Figure 42: First Sector of the File “About LE Robotics”

4 Observation

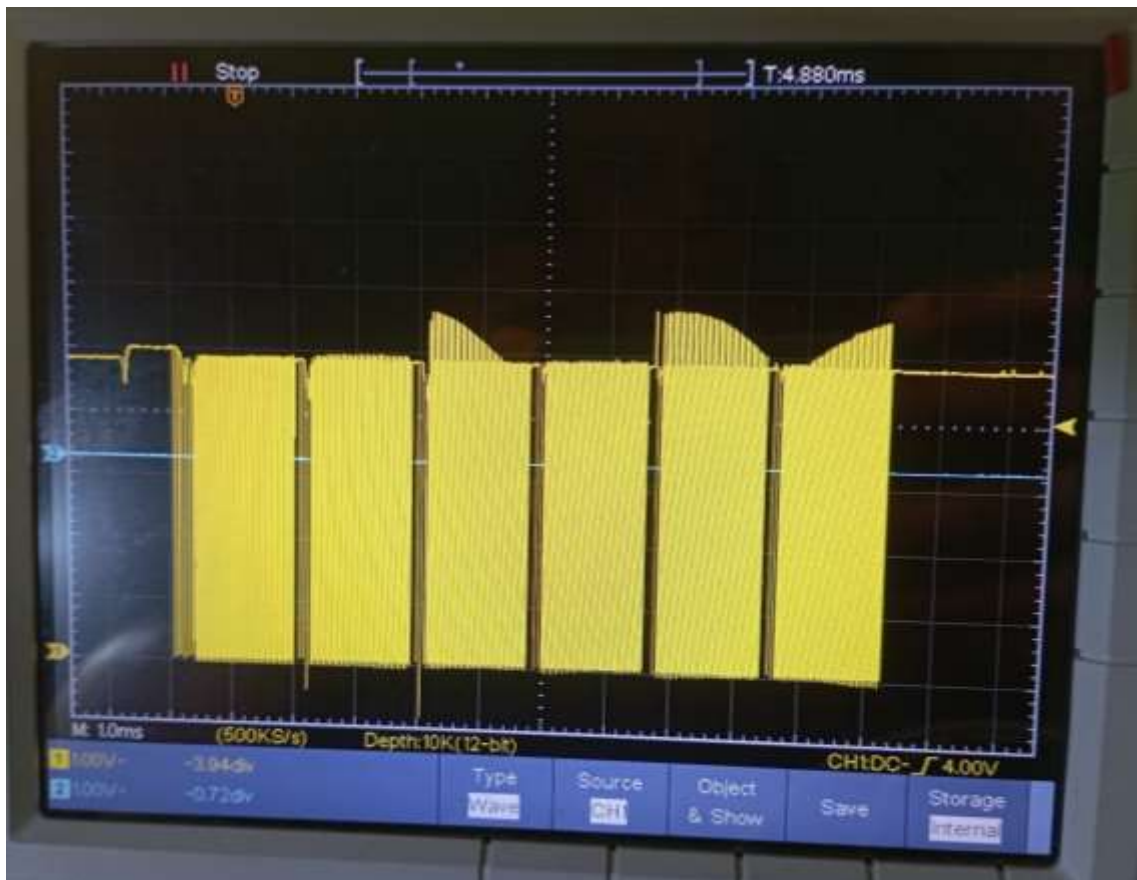


Figure 43: Observed Output from Oscilloscope

According to the Figure 43, reading speed is approximately 275 kBytes/Sec.

5 References

- [1] P. Stoffregen, "Understanding FAT32 Filesystems," 24 February 2005. [Online]. Available: <https://www.pjrc.com/tech/8051/ide/fat32.html>. [Accessed 16 Aug 2024].
- [2] S. L. Asanka P. Sayakkara Senior Lecturer at University of Colombo School of Computing (UCSC), "Diving into FAT file system with a Hex Editor," 08 November 2017. [Online]. Available: <https://recolog.blogspot.com/2017/11/diving-into-fat-file-system-with-hex.html>. [Accessed 16 08 2024].